

Raport Proiect PCBE: Game of Life

1. Descrierea Proiectului

Acest proiect implementează o simulare a “Jocului Vieții” (Game of Life) cu o abordare concurentă, unde fiecare celulă din simulare este un fir de execuție independent (**Thread**) care concurează pentru resurse limitate (hrană) pe o hartă comună.

Caracteristici principale:

- **Ciclu de viață:** Celulele gestionează propriul ciclu de viață, care include hrănirea, supraviețuirea, moartea și reproducerea. O celulă trebuie să consume hrană pentru a supraviețui; în caz contrar, moare de inaniție după o perioadă de timp.
- **Tipuri de celule:**
 - **AsexualCell:** Se reproduce prin diviziune, creând două celule noi în locații adiacente libere.
 - **SexualCell:** Necesită găsirea unui partener (o altă celulă sexuală) care dorește, de asemenea, să se reproducă.
- **Lumea partajată:** Harta (lumea) conține celule și unități de hrană, fiind o resursă partajată la care toate firele de execuție (celulele) trebuie să acceseze în mod sigur.

Obiectivul principal al proiectului este gestionarea corectă a concurenței și a accesului la starea partajată, folosind mecanisme de sincronizare din Java.

2. Membrii Echipei și Repository-ul Git

- **Membru:** Vlad Valean
- **Repository Git:** <https://github.com/Vlad-Valean/GameOfLife.git>

3. Descrierea Problemelor de Concurență Abordate

Aplicația abordează mai multe probleme clasice de concurență:

a) Managementul Stării Partajate (Shared State Management)

- **Problema:** Harta lumii, care conține pozițiile celulelor și a hranei, este o structură de date partajată, modificată constant de zeci sau sute de fire de execuție. Accesul nesincronizat ar duce la excepții (**ConcurrentModificationException**), stări inconsistente (de ex., două celule ocupând aceeași poziție) și “race conditions”.
- **Soluția:**
 - S-a utilizat `java.util.concurrent.ConcurrentHashMap` pentru a stoca celulele, hrana și celulele care așteaptă un partener. Această structură de date este “thread-safe” și oferă operații atomice precum `putIfAbsent`, `remove` și `computeIfAbsent`, care garantează că op-

erațiile compuse (verifică-dacă-există-apoi-adaugă) se execută ca o singură unitate, fără întreruperi.

- Cantitatea de hrană de la o anumită poziție este gestionată printr-un `java.util.concurrent.atomic.AtomicInteger`. Consumul de hrană se realizează printr-o buclă `compareAndSet` (CAS), o tehnică de sincronizare non-blocantă care evită utilizarea lock-urilor, îmbunătățind performanța.

b) Managementul Ciclului de Viață al Firelor de Execuție (Thread Lifecycle Management)

- **Problema:** Crearea și distrugerea manuală a firelor de execuție (`new Thread()`) pentru fiecare celulă este ineficientă și greu de gestionat, în special la o scară mare.
- **Soluția:**
 - Fiecare celulă implementează interfața `Runnable`.
 - Se folosește un `java.util.concurrent.ExecutorService` (în concret, `Executors.newCachedThreadPool()`) pentru a gestiona un “pool” de fire de execuție. Celulele noi (atât cele inițiale, cât și cele rezultate din reproducere) sunt trimise executorului prin metoda `submit()`. Acest lucru decuplează logica celulei de managementul direct al thread-urilor și permite reutilizarea eficientă a acestora.

c) Coordonarea între Fire de Execuție (Thread Coordination)

- **Problema:** Reproducerea celulelor sexuale necesită o coordonare complexă. O celulă trebuie să găsească un partener, iar acest proces trebuie să fie sigur și eficient, fără a intra în “deadlock” sau “livelock”.
- **Soluția:**
 - S-a implementat un mecanism de tip “wait/notify”. O celulă (`SexualCell`) care dorește să se reproducă, dar nu găsește un partener, se adaugă pe o listă de așteptare (`waitingGrid`) și intră în starea de așteptare folosind `wait()` în interiorul unui bloc `synchronized (this)`.
 - O altă celulă care caută activ un partener poate găsi o celulă în așteptare. Când o găsește, o scoate de pe lista de așteptare și o “trezește” apelând `notifyAll()` pe monitorul obiectului partener. Acest apel întrerupe starea de `wait()` a celulei adormite, permițându-i să continue execuția și să finalizeze procesul de împerechere.
 - Pentru a evita așteptarea la infinit, apelul `wait()` este temporizat (`wait(timeout)`).

4. Raportul de Implicare în Proiect

- **Vlad Valean:** 100%